Homework 2 Solutions

Alexander McLain

```
library(printr)
library(formatR)
library(kableExtra)
library(tidyverse)
library(rsample)
library(recipes)
library(yardstick)
library(caret)
library(broom)      # augment for residuals, leverage, cooks distance
library(lmtest)     # bptest for heteroscedasticity
library(car)        # vif, component+residual plots
library(glmnet)    # ridge/lasso
library(pls)       # partial least squares
library(lars)      # diabetes data (public health: diabetes progression)
library(ncvreg)    # MCP/SCAD
library(grpreg)    # grouped lasso
library(genlasso) # fused lasso / trend filtering
library(mlbench)
```

1. **(25 points)** For this exercise we'll use the "Communities and Crime Data Set". This data set contains variables related to violent crime. The goal is to predict the variable `ViolentCrimesPerPop`. The first five variables in the dataset should not be used in the regression models.

Reading in the data:

```
df.nomiss <- readRDS("community_crime.RDS")
```

This version of the data has the variables that have a lot of missingness removed. There are no missing values.

1. (5 pts) Fit a standard linear regression model predicting the per-capita violent crime rate from the community-level predictors. Report adjusted R^2 and comment on the fit.
2. (10 pts) Fit ridge and lasso regression models using cross-validation. Report the chosen penalty parameter , test error, and the number of predictors retained. Compare to the ordinary regression.
3. (10 pts) Fit MCP and SCAD regression models. Compare variable selection patterns to the lasso. Do MCP/SCAD retain fewer predictors while maintaining predictive accuracy? Briefly explain.

Part a

To do this, i'm going to start by splitting the data into training and test data:

```
set.seed(12345)
split_crime <- initial_split(df.nomiss, prop = 0.7, strata = NULL)
train <- training(split_crime)
test  <- testing(split_crime)
```

```r
X_tr <- as.matrix(dplyr::select(train, -ViolentCrime))
y_tr <- train$ViolentCrime
X_te <- as.matrix(dplyr::select(test,  -ViolentCrime))
y_te <- test$ViolentCrime


rmse <- function(truth, estimate) sqrt(mean((truth - estimate)^2))
```

```r
lm_model <- lm(ViolentCrime ~ ., data = train)
coef_table <- summary(lm_model)$coefficients
rownames(coef_table)  = gsub("[()]", "", rownames(coef_table) )   # removes both ( and )
colnames(coef_table) = c("Estimate","Std. Error","t value","P-value" )
coef_table %>% kable(digits = 3) %>% kable_styling(full_width = FALSE)
```

	Estimate	Std. Error	t value	P-value
Intercept	0.725	0.239	3.040	0.002
population	0.013	0.456	0.029	0.977
householdsize	0.005	0.099	0.047	0.962
racepctblack	0.170	0.061	2.772	0.006
racePctWhite	-0.061	0.071	-0.859	0.390
racePctAsian	-0.086	0.043	-2.018	0.044
racePctHisp	0.075	0.062	1.201	0.230
agePct12t21	0.068	0.121	0.564	0.573
agePct12t29	-0.263	0.176	-1.495	0.135
agePct16t24	-0.129	0.186	-0.695	0.487
agePct65up	0.080	0.118	0.674	0.501
numbUrban	-0.040	0.446	-0.090	0.928
pctUrban	0.034	0.018	1.878	0.061
medIncome	-0.218	0.200	-1.094	0.274
pctWWage	-0.241	0.104	-2.318	0.021
pctWFarmSelf	0.054	0.024	2.250	0.025
pctWInvInc	-0.206	0.079	-2.594	0.010
pctWSocSec	0.009	0.126	0.074	0.941
pctWPubAsst	0.052	0.053	0.973	0.331
pctWRetire	-0.066	0.043	-1.525	0.127
medFamInc	0.420	0.192	2.189	0.029
perCapInc	-0.088	0.226	-0.390	0.696
whitePerCap	-0.297	0.187	-1.584	0.113
blackPerCap	-0.036	0.029	-1.243	0.214
indianPerCap	-0.009	0.022	-0.403	0.687
AsianPerCap	0.005	0.022	0.223	0.823
OtherPerCap	0.024	0.022	1.088	0.277
HispPerCap	0.057	0.029	1.957	0.051
NumUnderPov	0.006	0.151	0.038	0.970
PctPopUnderPov	-0.143	0.072	-1.975	0.048
PctLess9thGrade	-0.130	0.079	-1.641	0.101
PctNotHSGrad	0.018	0.112	0.161	0.872
PctBSorMore	0.059	0.091	0.652	0.514
PctUnemployed	0.017	0.047	0.358	0.720
PctEmploy	0.305	0.091	3.345	0.001

PctEmplManu	-0.028	0.037	-0.750	0.453
PctEmplProfServ	0.013	0.048	0.271	0.787
PctOccupManu	0.064	0.063	1.013	0.311
PctOccupMgmtProf	0.076	0.099	0.768	0.443
MalePctDivorce	0.628	0.305	2.059	0.040
MalePctNevMarr	0.185	0.077	2.393	0.017
FemalePctDiv	0.297	0.402	0.740	0.459
TotalPctDiv	-0.942	0.659	-1.428	0.153
PersPerFam	-0.035	0.195	-0.179	0.858
PctFam2Par	0.072	0.186	0.387	0.699
PctKids2Par	-0.344	0.187	-1.838	0.066
PctYoungKids2Par	-0.043	0.056	-0.774	0.439
PctTeen2Par	-0.040	0.050	-0.808	0.419
PctWorkMomYoungKids	0.067	0.055	1.230	0.219
PctWorkMom	-0.163	0.063	-2.603	0.009
NumIlleg	-0.119	0.120	-0.993	0.321
PctIlleg	0.089	0.058	1.555	0.120
NumImmig	-0.169	0.085	-1.995	0.046
PctImmigRecent	0.023	0.050	0.469	0.639
PctImmigRec5	0.004	0.081	0.056	0.956
PctImmigRec8	0.086	0.095	0.906	0.365
PctImmigRec10	-0.064	0.071	-0.903	0.366
PctRecentImmig	-0.058	0.148	-0.390	0.696
PctRecImmig5	-0.251	0.274	-0.915	0.360
PctRecImmig8	0.061	0.337	0.182	0.856
PctRecImmig10	0.264	0.267	0.989	0.323
PctSpeakEnglOnly	-0.035	0.083	-0.428	0.669
PctNotSpeakEnglWell	-0.206	0.081	-2.534	0.011
PctLargHouseFam	0.085	0.263	0.323	0.747
PctLargHouseOccup	-0.244	0.278	-0.879	0.380
PersPerOccupHous	0.638	0.294	2.167	0.030
PersPerOwnOccHous	-0.209	0.203	-1.031	0.303
PersPerRentOccHous	-0.293	0.091	-3.209	0.001
PctPersOwnOccup	-0.616	0.427	-1.443	0.149
PctPersDenseHous	0.241	0.088	2.744	0.006
PctHousLess3BR	0.090	0.069	1.310	0.190
MedNumBR	0.020	0.023	0.867	0.386
HousVacant	0.111	0.081	1.366	0.172
PctHousOccup	-0.089	0.036	-2.450	0.014
PctHousOwnOcc	0.494	0.445	1.109	0.268
PctVacantBoarded	0.051	0.025	2.034	0.042
PctVacMore6Mos	-0.098	0.029	-3.324	0.001
MedYrHousBuilt	-0.003	0.033	-0.078	0.937
PctHousNoPhone	0.001	0.041	0.027	0.979
PctWOFullPlumb	0.004	0.024	0.152	0.879
OwnOccLowQuart	-0.416	0.247	-1.684	0.092
OwnOccMedVal	0.330	0.381	0.866	0.387
OwnOccHiQuart	0.062	0.203	0.306	0.759
RentLowQ	-0.204	0.078	-2.616	0.009
RentMedian	-0.121	0.187	-0.646	0.518

RentHighQ	-0.082	0.098	-0.834	0.404
MedRent	0.423	0.151	2.800	0.005
MedRentPctHousInc	0.046	0.038	1.199	0.231
MedOwnCostPctInc	-0.082	0.041	-1.983	0.048
MedOwnCostPctIncNoMtg	-0.080	0.028	-2.874	0.004
NumInShelters	0.167	0.072	2.309	0.021
NumStreet	0.176	0.053	3.326	0.001
PctForeignBorn	0.193	0.106	1.821	0.069
PctBornSameState	0.020	0.049	0.409	0.683
PctSameHouse85	-0.006	0.068	-0.096	0.924
PctSameCity85	0.016	0.044	0.365	0.715
PctSameState85	0.002	0.048	0.046	0.964
LandArea	0.056	0.060	0.928	0.354
PopDens	0.006	0.035	0.180	0.857
PctUsePubTrans	0.000	0.027	0.014	0.989
LemasPctOfficDrugUn	0.033	0.018	1.827	0.068

The R^2 of this model is 0.6957 (not shown). Here's some checks on the model fit.

Linearity: Component + Residual (Partial Residual) Plots

If relationships are strongly curved, consider transformations or adding terms (splines, polynomials).

```
car::crPlots(lm_model)  # opens a panel of C+R plots
```

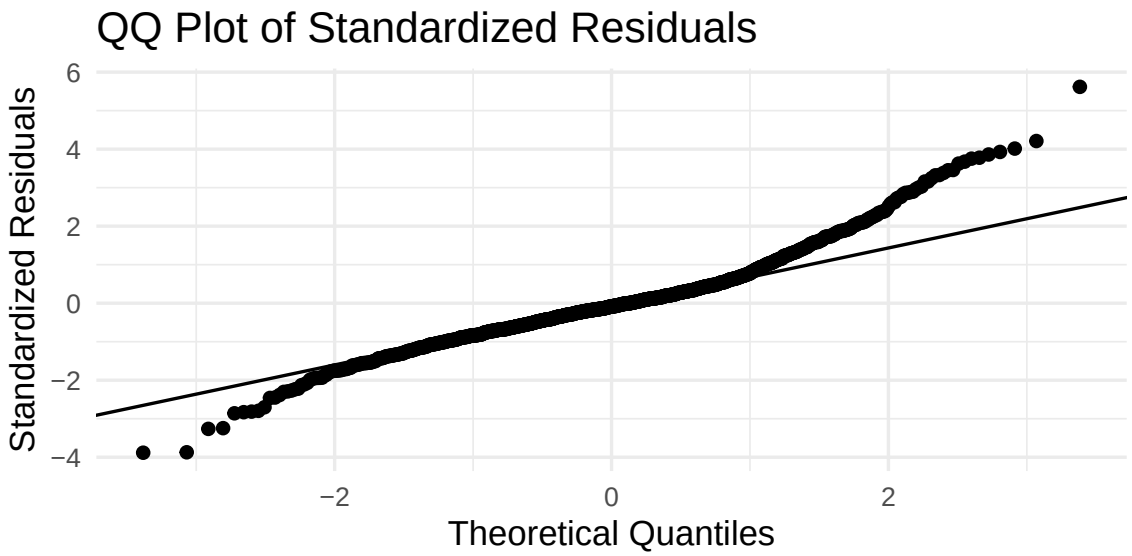
Not going to run this here, but when I checked the linearity looked OK.

Normality of Residuals (Primarily for CIs/p-values)

For prediction, normality is less critical than calibration and unbiasedness, but we inspect QQ plot.

```
aug <- augment(lm_model) %>% # augment gives us a lot of statistics that are useful.
  mutate(.stdres = rstandard(lm_model))  # standardized residuals


ggplot(aug, aes(sample = .stdres)) +
  stat_qq() + stat_qq_line() +
  labs(title = "QQ Plot of Standardized Residuals",
       x = "Theoretical Quantiles", y = "Standardized Residuals") +
  theme_minimal(base_size = 13)
```



The residuals don't look good here. The observed standardized residuals have much more variation than they should if the residuals were normally distributed.

What's the impact? Prediction intervals (for new observations) will not be valid. Having valid prediction intervals requires all the assumptions to be valid. The other impact would be on the type I error of the p-values for small to moderate sample sizes.

Homoscedasticity (Constant Error Variance)

We really don't need to look at this since we've already established that there are outliers in the data.

Multicollinearity (VIF)

We'll look at the top 10 VIF's (all way too large). Another sign of instability in our model.

```
vif_stats = car::vif(lm_model) %>% sort(decreasing = TRUE)
vif_stats[1:10]
```

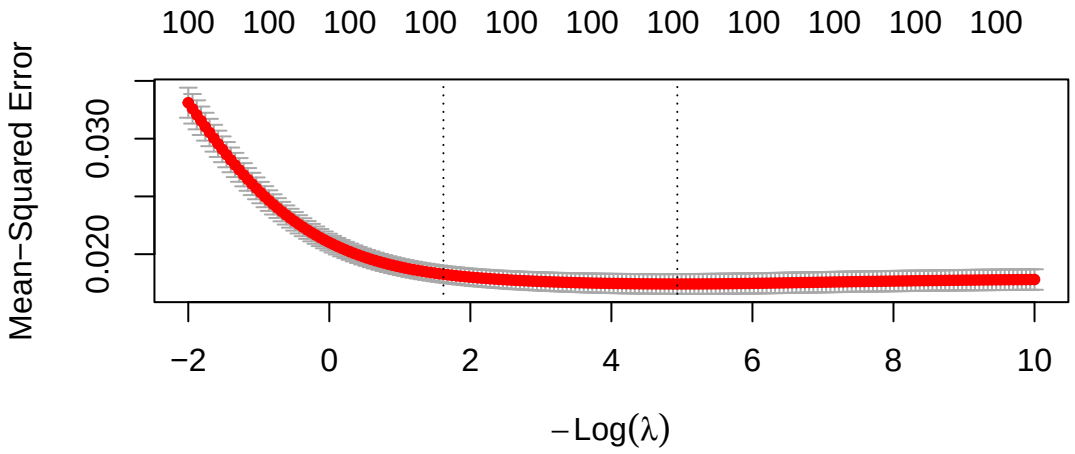
```
##     TotalPctDiv    OwnOccMedVal PctPersOwnOccup   PctHousOwnOcc    PctRecImmig8
##       1278.5425        672.8479        614.6002        589.3400        511.0568
##    FemalePctDiv      population    PctRecImmig5       numbUrban   PctRecImmig10
##        436.1913        341.5914        338.1451        332.4174        313.6804
```

Part B

We'll start with the ridge model:

```
set.seed(1123)

cv_ridge <- cv.glmnet(
  X_tr,
  y_tr,
  alpha = 0,
  standardize = TRUE,
  nfolds = 10,
  ## I usually don't specify lambda
  lambda = exp(seq(-10, 2, length.out = 200))
)
plot(cv_ridge)
```

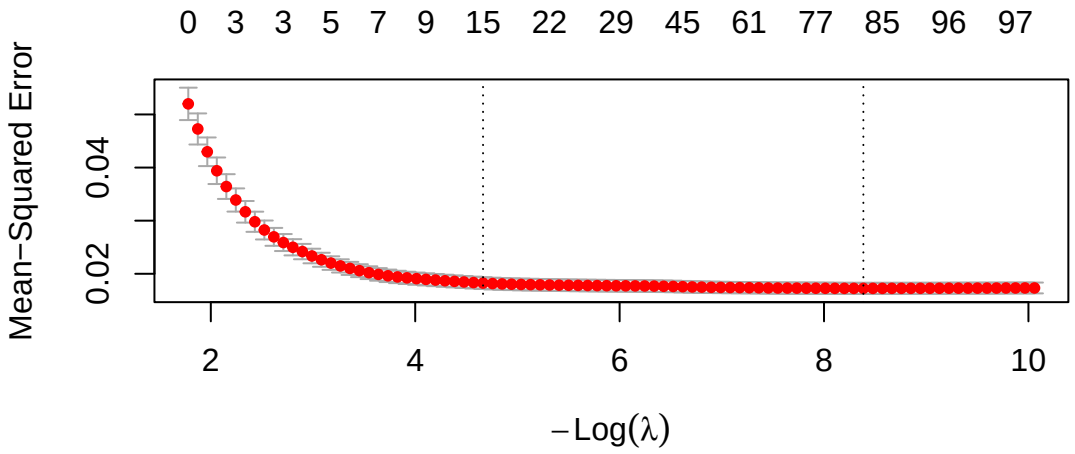



```
ridge_fit_min <- glmnet(
  X_tr,
  y_tr,
  alpha = 0,
  lambda = cv_ridge$lambda.min,
  standardize = TRUE
)

ridge_fit_1se <- glmnet(
  X_tr,
  y_tr,
  alpha = 0,
  lambda = cv_ridge$lambda.1se,
  standardize = TRUE
)
```

Lasso (L1)

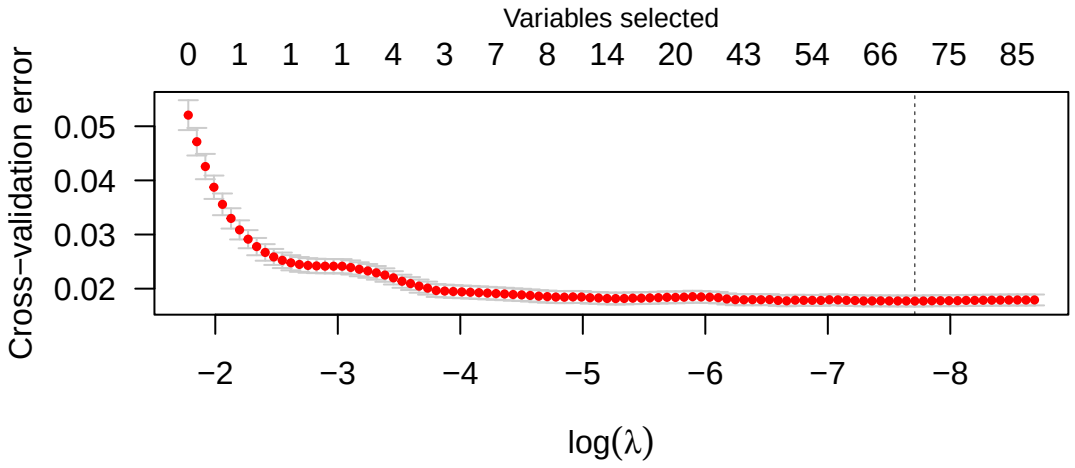
```
set.seed(2)
cv_lasso <- cv.glmnet(  X_tr, y_tr, alpha = 1, standardize = TRUE, nfolds = 10)
plot(cv_lasso)
```



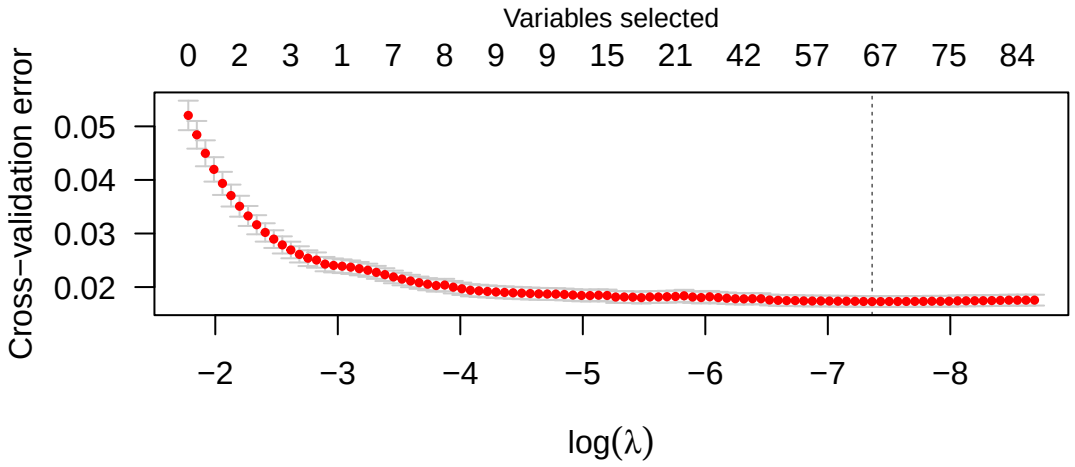

```
lasso_min  <-  glmnet(  X_tr,y_tr, alpha = 1, lambda = cv_lasso$lambda.min,  standardize = TRUE)
lasso_1se  <-  glmnet(  X_tr,y_tr, alpha = 1, lambda = cv_lasso$lambda.1se,  standardize = TRUE)
```

SCAD and MCP

```
set.seed(4)
cv_mcp  <- cv.ncvreg(X_tr,y_tr, penalty = "MCP",  family = "gaussian", nfolds = 10)
plot(cv_mcp)
```




```
cv_scad <- cv.ncvreg(X_tr,y_tr, penalty = "SCAD", family = "gaussian", nfolds = 10)
plot(cv_scad)
```




```
mcp_fit  <- cv_mcp$fit
scad_fit <- cv_scad$fit
```

Now let's look at the final beta coefficients:

```
### Beta Coefficients

# 1) Extract clean coefficient names
coef_names <- rownames(as.matrix(coef(ridge_fit_min)))

# 2) Build Beta tibble from pure numeric vectors (not matrices!)
Beta <- tibble(
  Variable  = coef_names,
  ridge_min = as.numeric(coef(ridge_fit_min)),
  ridge_1se = as.numeric(coef(ridge_fit_1se)),
  lasso_min = as.numeric(coef(lasso_min)),
  lasso_1se = as.numeric(coef(lasso_1se)),
  mcp       = as.numeric(mcp_fit$beta[, cv_mcp$min]),
  scad      = as.numeric(scad_fit$beta[, cv_scad$min])
)

Beta <- Beta |>
  dplyr::mutate(
    Variable = stringr::str_trunc(Variable, 10, side = "right")
  )

# 4) Round numeric values
Beta_disp <- Beta |>
  dplyr::mutate(dplyr::across(-Variable, ~round(.x, 3)))

Beta_disp %>% kable(digits = 3, caption = "Beta coefficients of penalized models.", escape = TRUE)
```

Table 2: Beta coefficients of penalized models.

Variable	ridge_min	ridge_1se	lasso_min	lasso_1se	mcp	scad
(Interc...	0.584	0.329	0.763	0.422	0.860	0.892
population	-0.036	0.009	-0.010	0.000	0.000	0.000
househo...	0.057	0.005	0.021	0.000	0.000	0.000
racepct...	0.149	0.082	0.179	0.035	0.182	0.177
racePct...	-0.076	-0.071	-0.056	-0.136	-0.064	-0.073
racePct...	-0.059	-0.010	-0.059	0.000	-0.071	-0.064
racePct...	0.044	0.008	0.069	0.000	0.096	0.086
agePct1...	0.003	-0.019	0.000	0.000	0.000	0.000
agePct1...	-0.193	-0.034	-0.313	-0.022	-0.363	-0.375
agePct1...	-0.034	-0.022	0.000	0.000	0.000	0.000
agePct65up	0.045	0.011	0.023	0.000	0.000	0.000
numbUrban	-0.022	0.014	0.000	0.000	0.000	0.000
pctUrban	0.032	0.018	0.031	0.015	0.031	0.032
medIncome	0.038	0.000	0.000	0.000	0.000	0.000
pctWWage	-0.097	-0.020	-0.216	0.000	-0.270	-0.280
pctWFar...	0.032	-0.008	0.045	0.000	0.055	0.056
pctWInvInc	-0.154	-0.045	-0.191	0.000	-0.208	-0.199
pctWSocSec	0.028	0.007	0.000	0.000	0.000	0.000
pctWPub...	0.050	0.028	0.036	0.000	0.037	0.034
pctWRetire	-0.065	-0.019	-0.070	0.000	-0.068	-0.074
medFamInc	0.035	-0.005	0.127	0.000	0.245	0.247
perCapInc	-0.063	-0.002	-0.010	0.000	0.000	0.000
whitePe...	-0.088	0.016	-0.243	0.000	-0.372	-0.388
blackPe...	-0.029	-0.014	-0.034	0.000	-0.038	-0.039
indianP...	-0.003	0.005	-0.004	0.000	-0.007	-0.006
AsianPe...	0.008	0.009	0.004	0.000	0.000	0.000
OtherPe...	0.017	0.010	0.020	0.000	0.024	0.022
HispPerCap	0.048	0.019	0.051	0.000	0.052	0.055
NumUnde...	-0.015	0.014	0.000	0.000	0.000	0.000
PctPopU...	-0.083	0.007	-0.121	0.000	-0.127	-0.137
PctLess...	-0.097	-0.006	-0.107	0.000	-0.119	-0.125
PctNotH...	0.007	0.014	0.000	0.000	0.000	0.000
PctBSor...	0.004	-0.010	0.025	0.000	0.000	0.000
PctUnem...	-0.015	0.010	0.000	0.000	0.000	0.000
PctEmploy	0.141	-0.004	0.234	0.000	0.279	0.287
PctEmpl...	-0.013	-0.012	-0.014	0.000	-0.031	-0.029
PctEmpl...	0.014	-0.004	0.015	0.000	0.000	0.000
PctOccu...	0.039	0.002	0.049	0.000	0.081	0.082
PctOccu...	0.032	-0.004	0.050	0.000	0.117	0.114
MalePct...	0.151	0.047	0.185	0.102	0.194	0.191
MalePct...	0.102	0.020	0.162	0.000	0.194	0.198
FemaleP...	-0.125	0.026	-0.235	0.000	-0.283	-0.281
TotalPc...	-0.033	0.034	0.000	0.000	0.000	0.000
PersPerFam	0.085	0.020	0.000	0.000	0.028	0.045
PctFam2Par	-0.050	-0.055	0.000	0.000	0.000	0.000
PctKids...	-0.131	-0.065	-0.248	-0.270	-0.282	-0.300
PctYoun...	-0.066	-0.052	-0.050	0.000	-0.039	-0.039
PctTeen...	-0.040	-0.055	-0.032	0.000	-0.041	-0.042
PctWork...	0.035	0.005	0.042	0.000	0.058	0.065
PctWorkMom	-0.111	-0.021	-0.134	-0.029	-0.151	-0.160

Variable	ridge_min	ridge_1se	lasso_min	lasso_1se	mcp	scad
NumIlleg	-0.061	0.042	-0.092	0.000	-0.119	-0.111
PctIlleg	0.118	0.078	0.101	0.196	0.083	0.080
NumImmig	-0.117	-0.011	-0.158	0.000	-0.170	-0.161
PctImmi...	0.014	0.000	0.011	0.000	0.040	0.000
PctImmi...	-0.020	0.000	0.000	0.000	-0.035	0.000
PctImmi...	0.051	0.011	0.057	0.000	0.092	0.052
PctImmi...	-0.023	0.010	-0.031	0.000	-0.054	0.000
PctRece...	-0.054	0.001	-0.069	0.000	-0.159	-0.072
PctRecI...	-0.027	0.005	-0.066	0.000	0.000	0.000
PctRecI...	0.037	0.010	0.000	0.000	0.000	0.000
PctRecI...	0.076	0.015	0.136	0.000	0.165	0.000
PctSpea...	-0.014	-0.001	-0.007	0.000	0.000	0.000
PctNotS...	-0.079	0.001	-0.146	0.000	-0.212	-0.198
PctLarg...	-0.029	0.024	0.000	0.000	-0.118	-0.119
PctLarg...	-0.057	0.017	-0.109	0.000	0.000	0.000
PersPer...	0.117	0.011	0.447	0.000	0.597	0.588
PersPer...	-0.118	-0.016	-0.251	0.000	-0.372	-0.372
PersPer...	-0.049	0.014	-0.146	0.000	-0.222	-0.226
PctPers...	-0.065	-0.015	-0.098	0.000	-0.146	-0.144
PctPers...	0.150	0.036	0.232	0.122	0.261	0.276
PctHous...	0.061	0.024	0.064	0.000	0.080	0.077
MedNumBR	0.005	-0.005	0.008	0.000	0.015	0.015
HousVacant	0.103	0.045	0.091	0.101	0.094	0.094
PctHous...	-0.102	-0.055	-0.102	-0.054	-0.101	-0.103
PctHous...	0.027	-0.005	0.000	0.000	0.000	0.000
PctVaca...	0.053	0.041	0.052	0.008	0.058	0.059
PctVacM...	-0.074	-0.017	-0.083	0.000	-0.094	-0.092
MedYrHo...	0.008	0.011	0.000	0.000	0.000	0.000
PctHous...	0.010	0.020	0.004	0.000	0.000	0.000
PctWOFu...	0.010	0.016	0.006	0.000	0.008	0.003
OwnOccL...	-0.050	-0.004	-0.126	0.000	-0.236	-0.242
OwnOccM...	0.014	0.001	0.000	0.000	0.000	0.000
OwnOccH...	0.047	0.005	0.118	0.000	0.220	0.226
RentLowQ	-0.134	-0.015	-0.203	0.000	-0.242	-0.239
RentMedian	0.001	-0.001	0.000	0.000	0.000	0.000
RentHighQ	0.021	0.004	-0.023	0.000	-0.122	-0.118
MedRent	0.105	0.008	0.230	0.000	0.368	0.362
MedRent...	0.061	0.030	0.054	0.000	0.059	0.058
MedOwnC...	-0.061	0.004	-0.078	0.000	-0.084	-0.090
MedOwnC...	-0.077	-0.026	-0.076	0.000	-0.078	-0.077
NumInSh...	0.142	0.054	0.146	0.000	0.168	0.159
NumStreet	0.180	0.079	0.185	0.129	0.184	0.185
PctFore...	0.092	0.014	0.172	0.000	0.217	0.281
PctBorn...	-0.003	-0.013	0.008	0.000	0.017	0.023
PctSame...	-0.003	0.001	0.000	0.000	0.000	0.000
PctSame...	0.022	0.015	0.011	0.000	0.013	0.001
PctSame...	0.008	0.001	0.006	0.000	0.000	0.000
LandArea	0.044	0.018	0.041	0.000	0.052	0.046
PopDens	0.010	0.012	0.004	0.000	0.010	0.001
PctUseP...	0.003	0.016	0.000	0.000	0.000	0.000
LemasPc...	0.040	0.031	0.037	0.017	0.038	0.039

Finally we can look at the test error and the number of non-zero coefficients.

```
pred_ols <- predict(lm_model, test)
ols_rmse = rmse(y_te, pred_ols)
pred_ridge_min <- predict(ridge_fit_min, X_te)
pred_ridge_1se <- predict(ridge_fit_1se, X_te)
rmse_min = rmse(y_te, pred_ridge_min)
rmse_1se = rmse(y_te, pred_ridge_1se)
pred_lasso_min <- predict(lasso_min, X_te)
pred_lasso_1se <- predict(lasso_1se, X_te)

lasso_min_rmse <- rmse(y_te, pred_lasso_min)
lasso_1se_rmse <- rmse(y_te, pred_lasso_1se)
pred_mcp  <- predict(mcp_fit,  X_te)
pred_scad <- predict(scad_fit, X_te)

mcp_rmse  <- rmse(y_te, pred_mcp)
scad_rmse <- rmse(y_te, pred_scad)


summ <- tibble(
  Method = c("OLS","Ridge (min)","Ridge (1SE)","Lasso (min)","Lasso (1SE)","MCP","SCAD"),
  Test_RMSE = c(
    ols_rmse,
    rmse_min,
    rmse_1se,
    lasso_min_rmse,
    lasso_1se_rmse,
    mcp_rmse,
    scad_rmse
  ),
  Nonzero = c(
    sum(coef(lm_model)[-1]!=0),
    sum(coef(ridge_fit_min )[-1]!=0),
    sum(coef(ridge_fit_1se )[-1]!=0),
    sum(coef(lasso_min )[-1]!=0),
    sum(coef(lasso_1se )[-1]!=0),
    sum(mcp_fit$beta[,cv_mcp$min][-1]!=0),
    sum(scad_fit$beta[,cv_scad$min][-1]!=0)
  )
)

summ %>% kable(digits = 4, caption = "Number of non-zero coefficients.", escape = TRUE)
```

Table 3: Number of non-zero coefficients.

Method	Test_RMSE	Nonzero
OLS	0.1482	100
Ridge (min)	0.1463	100
Ridge (1SE)	0.1503	100
Lasso (min)	0.1467	81
Lasso (1SE)	0.1474	15
MCP	0.1571	71
SCAD	0.1590	67

Here are the variables with a non-zero effect is the lasso model with the 1 SE rule:

```
tibble(
  Variables = coef_names[coef(lasso_1se )[,1]!=0][-1],
  Beta = coef(lasso_1se )[coef(lasso_1se )[,1]!=0,][-1]
) %>% kable(digits = 3)
```

Variables	Beta
racepctblack	0.035
racePctWhite	-0.136
agePct12t29	-0.022
pctUrban	0.015
MalePctDivorce	0.102
PctKids2Par	-0.270
PctWorkMom	-0.029
PctIlleg	0.196
PctPersDenseHous	0.122
HousVacant	0.101
PctHousOccup	-0.054
PctVacantBoarded	0.008
NumInShelters	0.000
NumStreet	0.129
LemasPctOfficDrugUn	0.017

I ran the above several times with different seeds for the initial creation of the training and test data. Commonly, OLS, Rigde (min) and Lasso (min) had the best RMSE with Lasso (1SE) close behind and MCP/SCAP quite a bit below the others.

Another common trend was for there to be a big gap in the number of predictors for Lasso with the min and 1SE criteria (we see that here).

For these results, I think it's clear that the Lasso (1SE) model is the best. It uses far fewer predictors (thus, has easier interpretations) and it's predictive ability is quite good.

2. The dataset parkinsons data is composed of a range of biomedical voice measurements from 42 people with early-stage Parkinson's disease recruited to a six-month trial of a telemonitoring device for remote symptom progression monitoring. The recordings were automatically captured in the patient's homes. Columns in the table contain subject number, subject age, subject gender, time interval from baseline recruitment date, motor UPDRS, total UPDRS, and 16 biomedical voice measures. The attributes of the data are

- subject number - Integer that uniquely identifies each subject
- age - Subject age
- sex - Subject gender '0' - male, '1' - female
- test_time - Time since recruitment into the trial. The integer part is the number of days since recruitment.
- motor_UPDRS - Clinician's motor UPDRS score, linearly interpolated
- total_UPDRS - Clinician's total UPDRS score, linearly interpolated

16 biomedical voice measures

- Jitter(%), Jitter(Abs), Jitter:RAP, Jitter:PPQ5, Jitter:DDP - Several measures of variation in fundamental frequency

- Shimmer, Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, Shimmer:APQ11, Shimmer:DDA - Several measures of variation in amplitude
- NHR, HNR - Two measures of ratio of noise to tonal components in the voice
- RPDE - A nonlinear dynamical complexity measure
- DFA - Signal fractal scaling exponent
- PPE - A nonlinear measure of fundamental frequency variation

Each row corresponds to one of 5,875 voice recording from these individuals. The main aim of the data is to predict the motor and total UPDRS scores ('motor_UPDRS' and 'total_UPDRS') from the 16 voice measures.

1. (5 pts) Fit a partial least squares (PLS) regression predicting **motor UPDRS** from the 16 biomedical voice measures. Use cross-validation to select the number of components.
2. (5 pts) Report the cross-validated prediction error and interpret how many components seem optimal. How do the results compare to lasso regression for the same outcome?
3. (5 pts) Fit a group lasso regression predicting **total UPDRS** from the grouped voice predictors.
4. (5 pts) Which groups are retained? How does grouped lasso differ from ordinary lasso in interpretation?

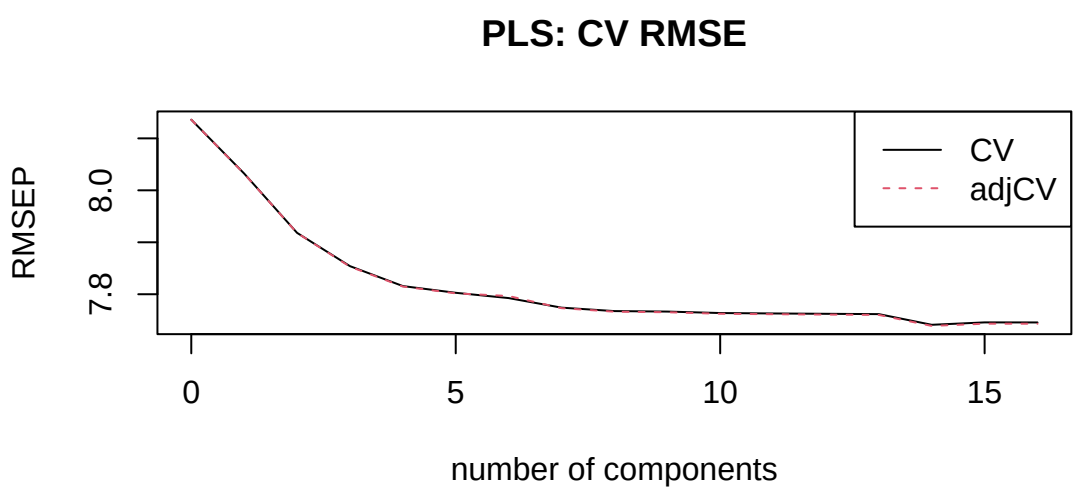
Part A

```
parkinson <- read_csv("parkinsons_updrs.txt", show_col_types = FALSE)
set.seed(3123121)

parkinson_voice <- parkinson %>%
  dplyr::select(
    motor_UPDRS, `Jitter(%)`:last_col()
  )
split_voice <- initial_split(parkinson_voice, prop = 0.7, strata = NULL)
train <- training(split_voice)
test  <- testing(split_voice)

X_tr <- as.matrix(dplyr::select(train, -motor_UPDRS))
y_tr <- train$motor_UPDRS
X_te <- as.matrix(dplyr::select(test,  -motor_UPDRS))
y_te <- test$motor_UPDRS


pls_fit <- plsr(motor_UPDRS ~ ., data = train, validation = "CV")
plot(RMSEP(pls_fit),legendpos ="topright", main ="PLS: CV RMSE")
```



Part B

```
# CV error of the best model (note this will be biased).
min(RMSEP(pls_fit)$val[estimate ="CV",,,drop = TRUE])
```

```
## [1] 7.741218
```

```
# Choose ncomp with lowest CV error
best_comp <-which.min(RMSEP(pls_fit)$val[estimate ="CV",,,drop = TRUE])
best_comp
```

```
## 14 comps
##       15
```

Now let's use this model to predict for the test data:

```
pred_pls <- predict(pls_fit, newdata = test, ncomp = best_comp)[,,1]
pls_rmse <- rmse(y_te, pred_pls)
```

We'll compare with lasso:

```
# lasso
set.seed(2)
cv_lasso <- cv.glmnet(X_tr, y_tr, alpha = 1, standardize = TRUE, nfolds = 10)
# Fit lasso models at .min and .1se
lasso_fit_min <- glmnet(X_tr, y_tr, alpha = 1, lambda = cv_lasso$lambda.min, standardize = TRUE)
lasso_fit_1se <- glmnet(X_tr, y_tr, alpha = 1, lambda = cv_lasso$lambda.1se, standardize = TRUE)
# Prediction
pred_lasso_min <- predict(lasso_fit_min, X_te)
pred_lasso_1se <- predict(lasso_fit_1se , X_te)
# RMSE
lasso_rmse_min <- rmse(y_te, pred_lasso_min)
lasso_rmse_1se <- rmse(y_te, pred_lasso_1se)
```

Let's see the winner!!

```
summ <- tibble(
  Method = c("PLS", "Lasso (min)", "Lasso (1SE)"),
  Test_RMSE = c(
```

```
    pls_rmse,
    lasso_rmse_min,
    lasso_rmse_1se
  ),
  Nonzero = c(
    best_comp-1, sum(coef(lasso_fit_min)[-1] != 0),
    sum(coef(lasso_fit_1se)[-1] != 0)
  )
) %>%
  arrange(Test_RMSE)
# PLS: number of components
summ %>%
  kable(digits = 3, caption = "PLS vs. Lasso on held-out test set") %>%
  kable_styling(full_width = FALSE)
```

Table 5: PLS vs. Lasso on held-out test set

Method	Test_RMSE	Nonzero
Lasso (min)	7.652	13
PLS	7.655	14
Lasso (1SE)	7.736	9

The lasso wins out, but PLS comes in second. Let's look at the coefficients:

```
tibble(
  Variable = c(colnames(X_tr)),
  PLS = as.vector(coef(pls_fit, ncomp = best_comp)[,1,1]),
  Lasso.min = as.vector(coef(lasso_fit_min)[-1]),
  Lasso.1se = as.vector(coef(lasso_fit_1se)[-1])
) %>% kable(digits = 3)
```

Variable	PLS	Lasso.min	Lasso.1se
Jitter(%)	32.480	27.324	0.000
Jitter(Abs)	-51680.239	-45253.407	-17970.609
Jitter:RAP	-43147.456	302.385	0.000
Jitter:PPQ5	112.022	19.993	0.000
Jitter:DDP	14491.382	0.836	0.000
Shimmer	83.518	0.000	0.000
Shimmer(dB)	-4.298	0.000	0.000
Shimmer:APQ3	24024.249	0.000	-0.007
Shimmer:APQ5	-190.321	-124.729	0.000
Shimmer:APQ11	126.531	110.154	11.095
Shimmer:DDA	-8031.124	-10.818	-3.885
NHR	-23.552	-19.163	-11.075
HNR	-0.459	-0.425	-0.273
RPDE	1.869	1.677	0.902
DFA	-28.014	-26.635	-21.380
PPE	19.302	19.207	17.458

Part C

First, we'll set out data up:

```r
set.seed(1234)

parkinson_UPDRS <- parkinson %>%
  dplyr::select(
    total_UPDRS, `Jitter(%)`:last_col()
  )
split_UPDRS <- initial_split(parkinson_UPDRS, prop = 0.7, strata = NULL)
train <- training(split_UPDRS)
test  <- testing(split_UPDRS)

X_tr <- as.matrix(dplyr::select(train, -total_UPDRS))
y_tr <- train$total_UPDRS
X_te <- as.matrix(dplyr::select(test,  -total_UPDRS))
y_te <- test$total_UPDRS


# group
grp_names <- c("Jitter(%)","Jitter(Abs)","Jitter:RAP","Jitter:PPQ5","Jitter:DDP",
               "Shimmer","Shimmer(dB)","Shimmer:APQ3","Shimmer:APQ5","Shimmer:APQ11",
               "Shimmer:DDA","NHR","HNR","RPDE","DFA","PPE")
gvec <- c(1,1,1,1,1, 2,2,2,2,2,2, 3,3, 4,4,4)
groups <- factor(gvec, labels = c("Jitter","Shimmer","Noise","Non-linear"))
X_tr_g <- as.matrix(X_tr[, grp_names])
X_te_g <- as.matrix(X_te[, grp_names])
```

Now we'll fit the grouped lasso:

```r
set.seed(412334)

cv_grp <- cv.grpreg(
  X_tr_g,
  y_tr,
  group = gvec,
  penalty = "grLasso",
  nfolds = 10,
  family = "gaussian")
#Fits the final group-lasso model at a specific penalty value lambda
grp_fit <- grpreg(
  X_tr_g,
  y_tr,
  group = gvec,
  penalty = "grLasso",
  lambda = cv_grp$lambda.min)

# Predictions on new data
pred_grp <- predict(grp_fit, X_te_g)
grp_rmse <- rmse(y_te, pred_grp)
grp_rmse %>% kable(digits = 4)
```

x
9.9942

Part D

Let's look at the coefficients:

```
tibble(
  Variable = c("Intercept",colnames(X_tr)),
  Group.Lasso = coef(grp_fit)
) %>% kable(digits = 3)
```

Variable	Group.Lasso
Intercept	60.344
Jitter(%)	391.387
Jitter(Abs)	-31690.303
Jitter:RAP	-46129.572
Jitter:PPQ5	-277.463
Jitter:DDP	15356.877
Shimmer	120.493
Shimmer(dB)	-6.925
Shimmer:APQ3	-9233.512
Shimmer:APQ5	-98.814
Shimmer:APQ11	95.322
Shimmer:DDA	3024.420
NHR	-34.577
HNR	-0.596
RPDE	4.375
DFA	-36.799
PPE	21.100

There are not many groups in this situation and all of them get chosen by the grouped lasso. As a result, the model ends up being more like a ridge regression than a lasso regression. The grouped lasso is best when there are lots of groups and the goal is to select some of them.

3. **(15 points)** This question will use the *Communities and Crime* dataset from question one.
   (a) (5 pts) Fit a GAM predicting violent crime rate using smooth functions of at least three predictors (choose variables you believe might have nonlinear effects).
   (b) (5 pts) Plot the smooth terms. Are any relationships clearly nonlinear?
   (c) (5 pts) Compare predictive performance of the GAM to a linear model with the same predictors. Which fits better?

Here, i'm going to use the same `split_crime` data that I created in question one. That way I can compare the test error with what I found there.

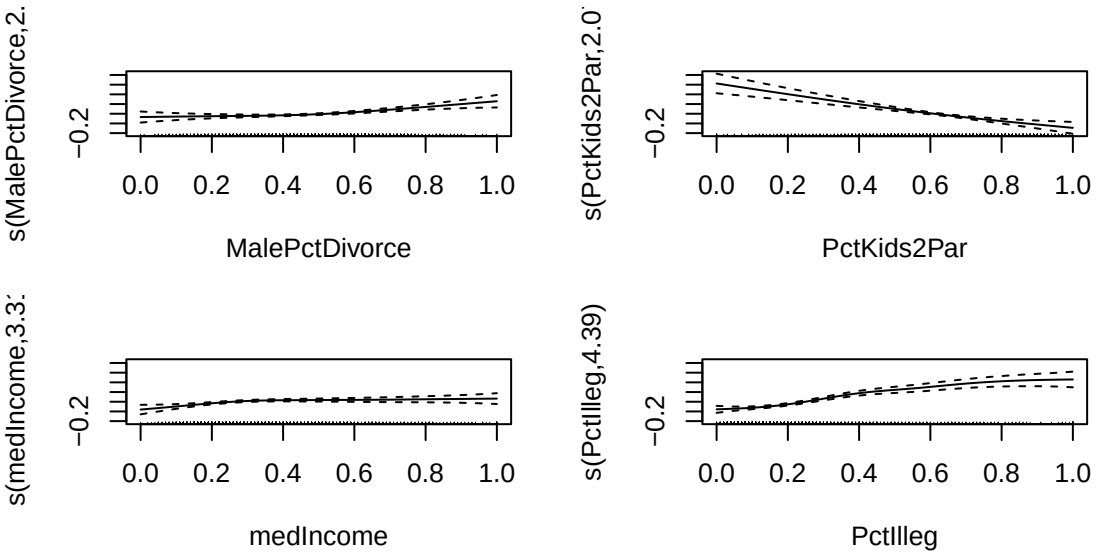
```
library(mgcv)
# get data
crime_smo <- dplyr::select(df.nomiss, c("ViolentCrime",))

train <- training(split_crime)
test <- testing(split_crime)

# GAM with penalized B-spline smooths (as in Example 2) and REML smoothing selection
gam_model<-gam(ViolentCrime ~ s(MalePctDivorce) +
                  s(PctKids2Par) +
                  s(medIncome) +
                  s(PctIlleg),
                data = train, method = "REML")
summary(gam_model)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## ViolentCrime ~ s(MalePctDivorce) + s(PctKids2Par) + s(medIncome) +
##     s(PctIlleg)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.232079   0.003853   60.23   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                       edf Ref.df      F  p-value
## s(MalePctDivorce) 2.619  3.343  8.047 1.45e-05 ***
## s(PctKids2Par)    2.067  2.689 16.164  < 2e-16 ***
## s(medIncome)      3.308  4.150  4.028  0.00274 **
## s(PctIlleg)       4.391  5.364 16.835  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.604   Deviance explained = 60.7%
## -REML = -697.14  Scale est. = 0.020715  n = 1395
```

```
plot(gam_model, pages = 1)
```



All 4 terms have a significant effect with non-linearities in their relationship with the outcome. Let's take a look at the predictions, and compare them with a linear only model, and to all the models that we found earlier.

```
### Getting the RMSE for the GAM model
pred_gam <- predict(gam_model, test)
gam_rmse  <- rmse(test$ViolentCrime, pred_gam)
```

```
### Getting the RMSE for the linear only model
lin_only_model<-lm(ViolentCrime ~ MalePctDivorce +
                   PctKids2Par +
                   medIncome +
                   PctIlleg,
                 data = train, method = "REML")
pred_lin_only <- predict(lin_only_model, test)
lin_only_rmse  <- rmse(test$ViolentCrime, pred_lin_only)


summ <- tibble(
  Method = c("OLS","Ridge (min)","Ridge (1SE)","Lasso (min)",
             "Lasso (1SE)","MCP","SCAD", "Gam",
             "OLS w 4 Linear Pred"),
  Test_RMSE = c(
    ols_rmse,
    rmse_min,
    rmse_1se,
    lasso_min_rmse,
    lasso_1se_rmse,
    mcp_rmse,
    scad_rmse,
    gam_rmse,
    lin_only_rmse
  ),
  Nonzero = c(
    sum(coef(lm_model)[-1]!=0),
    sum(coef(ridge_fit_min )[-1]!=0),
    sum(coef(ridge_fit_1se )[-1]!=0),
    sum(coef(lasso_min )[-1]!=0),
    sum(coef(lasso_1se )[-1]!=0),
    sum(mcp_fit$beta[,cv_mcp$min][-1]!=0),
    sum(scad_fit$beta[,cv_scad$min][-1]!=0),
    sum(gam_model$edf)-1,
    sum(coef(lin_only_model)[-1]!=0)
  )
)
summ %>% arrange(Test_RMSE) %>% kable(digits = 4)
```

Method	Test_RMSE	Nonzero
Ridge (min)	0.1463	100.000
Lasso (min)	0.1467	81.000
Lasso (1SE)	0.1474	15.000
OLS	0.1482	100.000
Ridge (1SE)	0.1503	100.000
Gam	0.1518	12.385
OLS w 4 Linear Pred	0.1534	4.000
MCP	0.1571	71.000
SCAD	0.1590	67.000

The GAM with 4 non-linear predictors does better than MCP/SCAD and the linear model with 4 linear predictors.

To estimate the number of coefficients for the GAM model I'm using the "effective degrees of freedom" summed over all coefficients. Here, this results in about 12 parameters (not counting the intercept), which is more than the lasso 1SE model, but less than all other models except the OLS with 4 linear predictors.

4. **(40 points)** The dataset "pima.indians.diabetes3.csv' ' contains data from 532 females who are at least 21 years of age and of Pima Indian heritage. The dataset consists of:

```
npregnant: Number of times pregnant
glucose: Plasma glucose concentration a 2 hours in an oral
glucose tolerance test
diastolic.bp: Diastolic blood pressure (mm Hg)
skinfold.thickness: Triceps skin fold thickness (mm)
bmi: Body mass index (weight in kg/(height in m)^2)
pedigree: Diabetes pedigree function
age: Age (years)
classdigit: Numerical class variable
class: Alphanumeric class variable
```

The purpose of this study was to determine whether the patient exhibited signs of diabetes mellitus according to World Health Organization criteria (i.e., if the 2-hour post-load plasma glucose level was at least 200 mg/dL at any survey examination or if it was detected during routine care). The two class variables indicate diabetes status using a blood test of 2-hour serum insulin. The goal is to use the other variables to predict diabetes status, as obtaining information on them is easier for patients, less invasive, and less expensive than a blood test. Use all other variables in each analysis.

Split the data into training (80%) and test (20%) sets

1. (5 pts) Fit a logistic regression with all predictors. Perform stepwise selection by AIC and BIC. Which predictors remain in each model?
2. (5 pts) Fit a logistic lasso model with cross-validation to select . Report the number of predictors retained with the "min" and "1se" values.
3. (10 pts) Using the test data, compare the predictive ability of the models found with AIC, BIC, and lasso in terms of AUC, specificity, sensitivity, PPV, and NPV. Which approach seems best in terms of prediction and interpretability?
4. (5 pts) Fit an LDA model and a QDA model to classify diabetes status.
5. (5 pts) Use cross-validation to estimate AUC, specificity, sensitivity, PPV, and NPV for each method.
6. (5 pts) Compare the performance of LDA, QDA, and logistic regression. Which would you prefer in practice, and why?
7. (5 pts) In the description of this study, it sounds like they are more interested in using this algorithm as a *screening* tool. Does this change which approach you think is best? Why or why not?

Part A

Next, we classify diabetes status using the Pima Indians Diabetes dataset.

```
data(PimaIndiansDiabetes2, package = "mlbench")
bin_df <- na.omit(PimaIndiansDiabetes2)
head(bin_df)
```

	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
4	1	89	66	23	94	28.1	0.167	21	neg
5	0	137	40	35	168	43.1	2.288	33	pos
7	3	78	50	32	88	31.0	0.248	26	pos
9	2	197	70	45	543	30.5	0.158	53	pos

	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
14	1	189	60	23	846	30.1	0.398	59	pos
15	5	166	72	19	175	25.8	0.587	51	pos

```
set.seed(42)
split <- initial_split(bin_df, prop = 0.8, strata = diabetes)
train <- training(split)
test  <- testing(split)

nrow(train); nrow(test)
```

```
## [1] 313
```

```
## [1] 79
```

Before commensing with the CV, we're going to create some functions to help us scale/center predictors and keep the outcome as a factor. This avoids leakage by **fitting the recipe inside resamples** when used with rsample.

```
rec_base <- recipe(diabetes ~ ., data = train) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_predictors())
prep_base <- prep(rec_base)          # for direct train fit (non-CV steps)
train_ready <- bake(prep_base, train)
test_ready  <- bake(prep_base, test)
```

```
full_glm <- glm(diabetes ~ ., data = train_ready, family = binomial())
null_glm <- glm(diabetes ~ 1, data = train_ready, family = binomial())

# AIC-based backwards (backwards starting at full model)
aic_fit <- MASS::stepAIC(full_glm, direction = "backward", trace = FALSE)

# BIC-based backwards (penalty k = log(n))
n_train <- nrow(train_ready)
#BIC gives the same model as AIC when backwards is used
#bic_fit <- MASS::stepAIC(full_glm, direction = "backward",
#                         k = log(n_train), trace = FALSE)

# BIC-based stepwise (penalty k = log(n))
bic_fit <- MASS::stepAIC(null_glm,
                         direction = "both",
                         k = log(n_train),
                         scope=list(lower=null_glm,
                                    upper=full_glm),
                         trace = FALSE)

# Summaries
tibble(
  AIC_parms = length(coef(aic_fit)),
  BIC_parms = length(coef(bic_fit)))
```

AIC_parms	BIC_parms
5	4

```r
formula(aic_fit)
```

```
## diabetes ~ glucose + mass + pedigree + age
```

```r
formula(bic_fit)
```

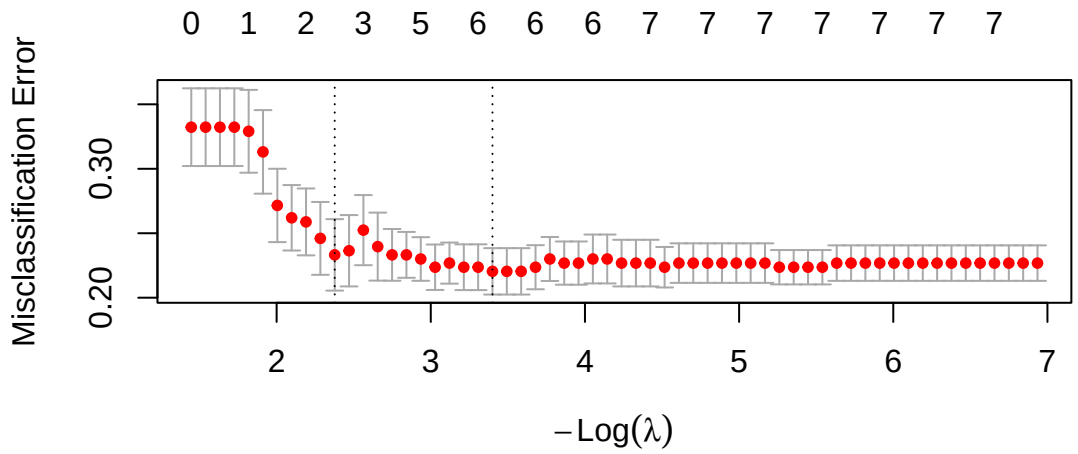
```
## diabetes ~ glucose + age + mass
```

Part B

```r
# Rebuild design matrices from the preprocessed data
x_tr <- model.matrix(diabetes ~ . - 1, data = train_ready)
y_tr <- train_ready$diabetes

x_te <- model.matrix(diabetes ~ . - 1, data = test_ready)
y_te <- test_ready$diabetes

set.seed(202)
cv_lasso <- cv.glmnet(x_tr, y_tr, alpha = 1, family = "binomial",
                      nfolds = 10, type.measure = "class")
plot(cv_lasso)
```




```r
lam_min  <- cv_lasso$lambda.min
lam_1se  <- cv_lasso$lambda.1se

fit_min <- glmnet(x_tr, y_tr, alpha = 1, family = "binomial", lambda = lam_min)
fit_1se <- glmnet(x_tr, y_tr, alpha = 1, family = "binomial", lambda = lam_1se)


tibble(
  Variable = rownames(coef(fit_min))[-1],
  coef_min = coef(fit_min)[-1],
  coef_1se = coef(fit_1se)[-1],
) %>% kable(digits = 3, caption = "Coefficient estimates for Logistic Lasso (note: all X and standardiz
```

Table 12: Coefficient estimates for Logistic Lasso (note: all X and standardized).

Variable	coef_min	coef_1se
pregnant	0.046	0.000
glucose	0.863	0.638
pressure	0.000	0.000
triceps	0.049	0.000
insulin	0.000	0.000
mass	0.225	0.000
pedigree	0.078	0.000
age	0.264	0.067

Inspect sparsity:

```
nnz <- function(m) sum(as.numeric(m)!=0)
tibble(
  Model = c("Lasso (min)","Lasso (1SE)"),
  Nonzero = c(nnz(coef(fit_min)), nnz(coef(fit_1se)))
) %>% kable()
```

Model	Nonzero
Lasso (min)	7
Lasso (1SE)	3

Part C Predict for the test data

```
p_min <- predict(fit_min, newx = x_te, type = "response")[,1]
p_1se <- predict(fit_1se, newx = x_te, type = "response")[,1]

cls_min <- factor(ifelse(p_min >= 0.5,"pos", "neg"), levels = levels(y_te))
cls_1se <- factor(ifelse(p_1se >= 0.5,"pos", "neg"), levels = levels(y_te))


lasso_res <- tibble(
  model = c("Lasso (min)","Lasso (1SE)"),
  AUC   = c(roc_auc_vec(y_te, p_min, event_level = "second"),
            roc_auc_vec(y_te, p_1se, event_level = "second")),
  Sens  = c(sens_vec(y_te, cls_min, event_level = "second"),
            sens_vec(y_te, cls_1se, event_level = "second")),
  Spec  = c(spec_vec(y_te, cls_min, event_level = "second"),
            spec_vec(y_te, cls_1se, event_level = "second")),
  NPV   = c(npv_vec(y_te, cls_min, event_level = "second"),
            npv_vec(y_te, cls_1se, event_level = "second")),
  PPV   = c(ppv_vec(y_te, cls_min, event_level = "second"),
            ppv_vec(y_te, cls_1se, event_level = "second"))
)

lasso_res %>% kable(digits = 3, caption = "Test-set performance: Logistic Lasso")
```

Table 14: Test-set performance: Logistic Lasso

model	AUC	Sens	Spec	NPV	PPV
Lasso (min)	0.911	0.462	0.981	0.788	0.923
Lasso (1SE)	0.863	0.269	1.000	0.736	1.000

Part D

LDA and QDA

We'll fit LDA and QDA (after standardizing predictors). QDA can have singularity issues when classes are small; standardization and all-numeric predictors help.

```
# Train LDA/QDA on preprocessed training data
lda_fit <- MASS::lda(diabetes ~ ., data = train_ready)
qda_fit <- MASS::qda(diabetes ~ ., data = train_ready)

# Predict on test set
pred_lda <- predict(lda_fit, newdata = test_ready)
pred_qda <- predict(qda_fit, newdata = test_ready)

head(pred_lda$posterior)
```

neg	pos
0.2028106	0.7971894
0.2677601	0.7322399
0.6737562	0.3262438
0.2542996	0.7457004
0.6265085	0.3734915
0.8824833	0.1175167

```
head(pred_lda$class)
```

```
## [1] pos pos neg pos neg neg
## Levels: neg pos
```

```
# yardstick expects class probs for AUC; LDA/QDA return posteriors
p_lda <- pred_lda$posterior[, "pos"]
p_qda <- pred_qda$posterior[, "pos"]

cls_lda <- factor(pred_lda$class, levels = levels(test_ready$diabetes))
cls_qda <- factor(pred_qda$class, levels = levels(test_ready$diabetes))

LQDA_res <- tibble(
  model = c("LDA","QDA"),
  AUC   = c(roc_auc_vec(test_ready$diabetes, p_lda, event_level = "second"),
            roc_auc_vec(test_ready$diabetes, p_qda, event_level = "second")),
  Sens  = c(sens_vec(test_ready$diabetes, cls_lda, event_level = "second"),
            sens_vec(test_ready$diabetes, cls_qda, event_level = "second")),
  Spec  = c(spec_vec(test_ready$diabetes, cls_lda, event_level = "second"),
            spec_vec(test_ready$diabetes, cls_qda, event_level = "second")),
  NPV   = c(npv_vec(test_ready$diabetes,  cls_lda, event_level = "second"),
            npv_vec(test_ready$diabetes,  cls_qda, event_level = "second")),
```

```
  PPV   = c(ppv_vec(test_ready$diabetes,  cls_lda, event_level = "second"),
            ppv_vec(test_ready$diabetes,  cls_qda, event_level = "second"))
)
LQDA_res %>% kable(digits = 3, caption = "Test-set performance: LDA and QDA")
```

Table 16: Test-set performance: LDA and QDA

model	AUC	Sens	Spec	NPV	PPV
LDA	0.920	0.462	1.000	0.791	1.000
QDA	0.888	0.615	0.906	0.828	0.762

Part E

Here is a script to get CV estimated metrics from each of the methods:

```
# Set up cross-validation control
ctrl <- trainControl(
  method = "cv",                  # k-fold CV
  number = 10,                    # 10-fold
  classProbs = TRUE,              # needed for AUC
  summaryFunction = twoClassSummary,  # carets built-in ROC/Sens/Spec summary
  savePredictions = "final"       # save out-of-fold preds
)

#----------------------#
# 1. LDA
#----------------------#
set.seed(735)
fit_lda <- train(
  diabetes ~ .,
  data = train,
  method = "lda",
  metric = "ROC",
  trControl = ctrl
)

#----------------------#
# 2. QDA
#----------------------#
set.seed(735)
fit_qda <- train(
  diabetes ~ .,
  data = train,
  method = "qda",
  metric = "ROC",
  trControl = ctrl
)

#----------------------#
# 3. Logistic LASSO (glmnet)
#----------------------#
# caret automatically tunes alpha (lasso/ridge) and lambda
set.seed(735)
```

```r
fit_lasso <- train(
  diabetes ~ .,
  data = train,
  method = "glmnet",
  metric = "ROC",
  tuneGrid = expand.grid(alpha = 1, lambda = exp(seq(-5, 1, length = 100))),
  trControl = ctrl
)

#----------------------#
# Extract cross-validated performance metrics
#----------------------#

res <- resamples(list(LDA = fit_lda, QDA = fit_qda, LASSO = fit_lasso))
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: LDA, QDA, LASSO
## Number of resamples: 10
##
## ROC
##            Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## LDA   0.7748918 0.7880952 0.8048701 0.8270779 0.8785714 0.8904762    0
## QDA   0.7012987 0.7750000 0.8028139 0.8045887 0.8595238 0.8904762    0
## LASSO 0.7662338 0.7857143 0.8022727 0.8290260 0.8801948 0.9047619    0
##
## Sens
##            Min.   1st Qu.    Median      Mean   3rd Qu.     Max. NA's
## LDA   0.7619048 0.8517857 0.8571429 0.8707143 0.9047619 1.000000    0
## QDA   0.7619048 0.8095238 0.8535714 0.8469048 0.8571429 0.952381    0
## LASSO 0.7619048 0.8571429 0.9023810 0.8804762 0.9047619 1.000000    0
##
## Spec
##       Min.   1st Qu.    Median      Mean   3rd Qu. Max. NA's
## LDA    0.2 0.5454545 0.5727273 0.5672727 0.6272727  0.8    0
## QDA    0.4 0.4659091 0.5727273 0.5863636 0.6840909  0.9    0
## LASSO  0.2 0.4659091 0.5727273 0.5490909 0.6272727  0.8    0
```

```r
# carets resamples() summary gives ROC, Sens, Spec across CV folds.

#----------------------#
# Calculate additional metrics (PPV & NPV) manually
#----------------------#

get_metrics <- function(model, data) {
  pred <- model$pred
  # keep only rows matching best tune parameters
  if ("lambda" %in% names(pred)) {
    best <- model$bestTune
    pred <- dplyr::inner_join(pred, best, by = intersect(names(best), names(pred)))
  }
```

```
  cm <- caret::confusionMatrix(pred$pred, pred$obs, positive = "pos")

  # Extract standard metrics
  roc_obj <- pROC::roc(pred$obs, pred$pos, levels = c("neg", "pos"))
  auc_val <- pROC::auc(roc_obj)

  tibble(
    Model = model$method,
    AUC = auc_val,
    Sensitivity = cm$byClass["Sensitivity"],
    Specificity = cm$byClass["Specificity"],
    PPV = cm$byClass["Pos Pred Value"],
    NPV = cm$byClass["Neg Pred Value"]
  )
}

results <- rbind(
  get_metrics(fit_lda, train),
  get_metrics(fit_qda, train),
  get_metrics(fit_lasso, train)
)

results %>% kable(digits = 3)
```

Model	AUC	Sensitivity	Specificity	PPV	NPV
lda	0.8174457	0.567	0.871	0.686	0.802
qda	0.8040578	0.587	0.847	0.656	0.805
glmnet	0.8191940	0.548	0.880	0.695	0.797

Part F

To compare the results, we could use what's above, or the test results we obtained earlier (shown below):

```
bind_rows(lasso_res,LQDA_res) %>% kable(digits = 3, caption = "Test-set performance: Lasso, LDA, and QDA
```

Table 18: Test-set performance: Lasso, LDA, and QDA

model	AUC	Sens	Spec	NPV	PPV
Lasso (min)	0.911	0.462	0.981	0.788	0.923
Lasso (1SE)	0.863	0.269	1.000	0.736	1.000
LDA	0.920	0.462	1.000	0.791	1.000
QDA	0.888	0.615	0.906	0.828	0.762

Part G

When we are interested in screening, we would like a method that has better sensitivity so that we don't miss as many true diseased individuals. Using the results from Part E or F, QDA has the best sensitivity. As a result, it would be the best method to use if screening was our main goal.